

Comprendre la séparabilité de 5 cancers dans des données génétiques à très haute dimension

Guide d'implémentation pas à pas, écrit pour quelqu'un qui débute en science des données. Chaque étape du code est expliquée avec l'intuition, la formule mathématique exacte, un diagramme, et les résultats obtenus sur les vraies données.

801 patients

20 531 gènes

5 types de cancer

Python · numpy / pandas / scipy / scikit-learn

SOMMAIRE

1. [Le problème posé](#)
2. [Étape 0 — Préparer les données](#)
3. [Méthode 1 — Mesurer sans regarder](#)
4. [Méthode 2a — Choisir 2 bons gènes](#)
5. [Méthode 2b — ACP](#)
6. [Méthode 2c — t-SNE](#)
7. [Analyse comparative](#)
8. [Limites de la mesure Overlap](#)
9. [Organisation du code](#)
10. [Glossaire](#)
11. [Sources](#)

1 LE PROBLÈME POSÉ

801 patients, 20 531 gènes, 5 cancers : sont-ils vraiment "séparés" ?

Le jeu de données TCGA-PANCAN-HiSeq-801×20531 décrit 801 patients atteints de cancer. Pour chaque patient, on connaît le niveau d'activité (l'*expression*) de 20 531 gènes — autrement dit, chaque patient est un point dans un espace à 20 531 dimensions. On connaît aussi son diagnostic : l'un de 5 types de cancer.

● BRCA — 300 patients ■ KIRC — 146 patients ▲ LUAD — 141 patients ◆ PRAD — 136 patients

▼ COAD — 78 patients

Question du TP : ces 5 groupes forment-ils des "nuages" de points bien distincts dans cet espace, ou se chevauchent-ils ? On y répond de deux façons complémentaires, qui structurent tout le reste du projet :

M1 MÉTHODE 1 — SANS DESSINER

Une formule mathématique unique (Overlap) qui résume, par un seul nombre, si deux classes se chevauchent — utilisable même quand on a bien trop de dimensions pour dessiner quoi que ce soit.

M2 MÉTHODE 2 — EN DESSINANT

On réduit intelligemment à 2 dimensions (bons gènes choisis, ACP, ou t-SNE) pour pouvoir *voir* les classes sur un nuage de points, et on vérifie l'intuition visuelle avec le même score Overlap.

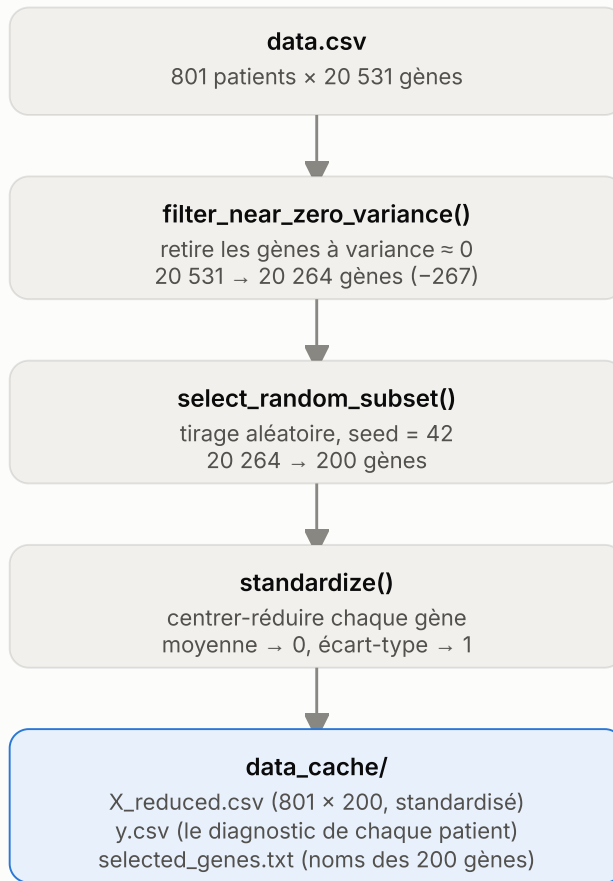
REMARQUE DE TRANSPARENCE

L'énoncé mentionne 16 382 variables, mais le fichier réel data.csv en contient 20 531 (probablement une version différente du jeu de données). Cela ne change rien à la démarche : elle s'applique quel que soit le nombre de gènes de départ.

2 ÉTAPE 0 — PRÉPARER LES DONNÉES

Pourquoi on ne peut pas juste "lancer les calculs" sur 20 531 gènes

Deux opérations plus loin dans le TP — l'ACP et la distance de Mahalanobis — ont besoin d'une **matrice de covariance**, une table qui dit à quel point chaque paire de gènes varie ensemble. Avec 20 531 gènes, cette matrice a $20531 \times 20531 \approx 421$ millions de cases, et il faut ensuite l'*inverser* pour Mahalanobis — numériquement instable et très lourd. Il faut donc réduire le nombre de variables *avant* de commencer.



Pipeline de préparation des données — `src/data_loading.py` + `src/build_dataset.py`. Chaque étape réduit ou transforme la matrice avant de la sauvegarder.

2.1 – Filtrer les gènes constants

Un gène dont la valeur est identique chez tous les 801 patients ne permet de distinguer personne : c'est du bruit pur pour la suite. On calcule la variance de chaque colonne (chaque gène) et on ne garde que celles au-dessus d'un seuil minuscule.

$$\text{Var}(\text{gène}_j) = (1/(n-1)) \cdot \sum_i (x_{ij} - \text{moyenne}_j)^2$$

n = 801 patients. Pandas utilise $n-1$ au dénominateur (variance "d'échantillon"). Résultat réel : 267 gènes ont une variance exactement nulle et sont retirés → 20 264 gènes restants.

```
src/data_loading.py

def filter_near_zero_variance(X, threshold=1e-8):
    variances = X.var(axis=0)
    keep = variances[variances > threshold].index
    return X[keep]
```

2.2 – Tirer 200 gènes au hasard (et pas les "meilleurs")

On pourrait choisir à l'avance les gènes qui séparent le mieux les cancers — mais cela biaiserait artificiellement la Méthode 1, qui a justement pour but de tester si un ensemble de variables *quelconque* sépare bien les classes. On tire donc 200 gènes au hasard parmi les 20 264 restants, avec une graine (seed) fixée à 42 pour que le tirage soit reproductible.

POURQUOI 200 ET PAS PLUS

La distance de Mahalanobis (section suivante) exige d'inverser une matrice de covariance de taille $p \times p$. Avec 801 patients, il faut p nettement plus petit que 801 pour que cette matrice reste inversible de façon stable. À $p = 200$, le ratio échantillons/variables est $801/200 \approx 4\times$, et le nombre de conditionnement mesuré de la matrice de covariance est ≈ 625 — acceptable, bien qu'un ratio de $8\times$ ($p = 100$) aurait été plus confortable.

2.3 – Standardiser (centrer-réduire)

Les niveaux d'expression génique n'ont pas du tout la même échelle d'un gène à l'autre (certains varient entre 0 et 5, d'autres entre 0 et 5000). Sans standardisation, les gènes à grande échelle domineraient artificiellement toutes les distances. On transforme chaque gène en *score-z* : combien d'écarts-types au-dessus ou en-dessous de la moyenne se trouve chaque valeur.

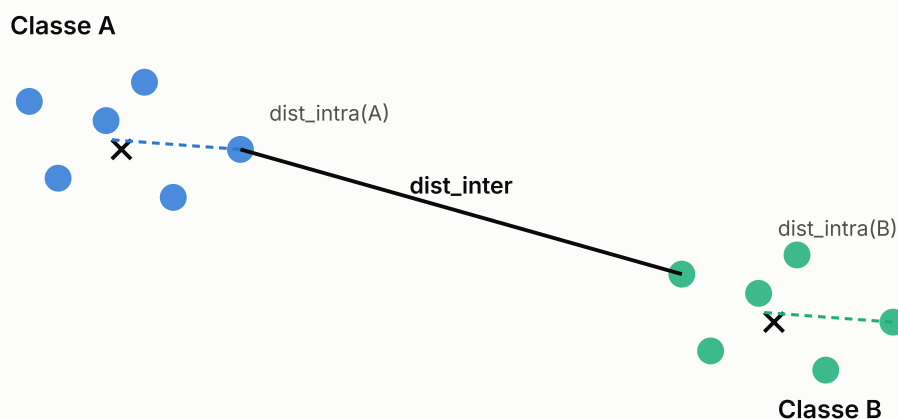
$$z_{ij} = (x_{ij} - \text{moyenne}_j) / \text{écart-type}_j$$

Après cette transformation, chaque gène a moyenne ≈ 0 et écart-type ≈ 1 — vérifié dans le code (moyenne résiduelle $\approx 1e-17$, un zéro numérique).

Ces trois étapes sont enchaînées par `build_dataset.py`, qui affiche la taille des données à chaque étape et sauvegarde le résultat final dans `data_cache/` — et surtout *pas* les données brutes des patients, seulement la liste des 200 noms de gènes retenus, conformément à la consigne de ne pas soumettre de données personnelles/sensibles.

Un seul nombre pour dire si deux classes se chevauchent

Impossible de dessiner un nuage de points en 200 dimensions. La Méthode 1 remplace le dessin par trois quantités calculées mathématiquement : à quel point chaque classe est "étalée" (intra-classe), à quel point deux classes sont "proches" (inter-classe), et le rapport des deux (Overlap).



dist_intra = le point le plus *loin* du centre de sa propre classe (la "taille" du nuage). dist_inter = la plus courte distance entre les deux nuages (via leurs centres). Plus dist_inter est grand par rapport aux deux dist_intra , moins les classes se chevauchent.

3.1 – Les trois formules

$$\text{dist_intra}(C) = \max \{ d(x_i, x_C) \mid x_i \in C \}$$

Distance du patient le plus éloigné au centre (la moyenne) de sa propre classe C.

$$\text{dist_inter}(C1, C2) = \min(\text{dist}(C1 \rightarrow C2), \text{dist}(C2 \rightarrow C1))$$

$$\text{où } \text{dist}(C1 \rightarrow C2) = \min \{ d(x_i, x_{C2}) \mid x_i \in C1 \}$$

On prend le plus petit des deux sens : le patient de C1 le plus proche du centre de C2, et vice-versa — puis on garde le minimum des deux.

$$\text{Overlap}(C1, C2) = (\text{dist_intra}(C1) + \text{dist_intra}(C2)) / (2 \times \text{dist_inter}(C1, C2))$$

$\text{Overlap} < 1$ est une condition *suffisante* (mais pas nécessaire) pour dire "ces deux classes sont bien séparées".

3.2 – Deux façons de mesurer une distance

Le code teste les trois formules ci-dessus avec deux définitions différentes de "distance" :

Euclidienne

$$d(x,y) = \sqrt{ \sum_j (x_j - y_j)^2 }$$

Traite chaque gène indépendamment, à poids égal. Simple, mais ignore que des gènes peuvent être corrélés entre eux.

Mahalanobis

$$d(x,y) = \sqrt{ (x-y)^t \cdot \Sigma^{-1} \cdot (x-y) }$$

Σ est la matrice de covariance ; Σ^{-1} "corrige" la distance pour ne pas compter deux fois l'information de deux gènes très corrélés. Calculée **une seule fois**, de façon globale sur les 801 patients (pas par classe) — la plus petite classe (COAD, 78 patients) est trop petite pour estimer sa propre covariance sur 200 variables.

3.3 – Ce que fait le code

```
src/distances.py

def dist_intra(X, metric="euclidean", VI=None):
    center = class_center(X)                # moyenne des lignes
    return cdist(X, center, metric=metric, VI=VI).max()

def dist_inter(X1, X2, metric="euclidean", VI=None):
    d_1_to_c2 = cdist(X1, class_center(X2), metric=metric, VI=VI).min()
    d_2_to_c1 = cdist(X2, class_center(X1), metric=metric, VI=VI).min()
    return min(d_1_to_c2, d_2_to_c1)

def overlap(intra1, intra2, inter):
    if inter == 0:
        return np.inf                # chevauchement total, pas une erreur
    return (intra1 + intra2) / (2 * inter)
```

`pairwise_overlap()` répète ce calcul pour les 10 paires possibles parmi les 5 classes, et renvoie une matrice 5×5 symétrique.

COMMENT ON A VÉRIFIÉ QUE LE CODE EST CORRECT

1) Un exemple à la main en 2D (3 points par classe) a donné les mêmes résultats que le code, au centième près. 2) Sur des données synthétiques bien séparées en 200D, le code trouve bien $\text{Overlap} = 0.072 (< 1)$ — donc il *peut* détecter une vraie séparation. 3) Le nombre de conditionnement de la matrice de covariance (≈ 625) confirme que l'inversion pour Mahalanobis est numériquement stable.

3.4 – Résultats sur les 200 gènes aléatoires

CLASSE	INTRA (EUCLIDIENNE)	INTRA (MAHALANOBIS)
BRCA	26.13	21.43
COAD	27.52	25.62
KIRC	28.19	20.45
LUAD	27.93	24.60
PRAD	32.74	26.09

OVERLAP (EUCLIDIENNE)	BRCA	COAD	KIRC	LUAD	PRAD
BRCA	—	2.337	2.104	2.581	2.655
COAD	2.337	—	2.062	2.744	2.351
KIRC	2.104	2.062	—	2.415	2.406
LUAD	2.581	2.744	2.415	—	2.857
PRAD	2.655	2.351	2.406	2.857	—

En vert : la paire la moins chevauchée. En rouge : la plus chevauchée. Toutes les valeurs restent > 1 .

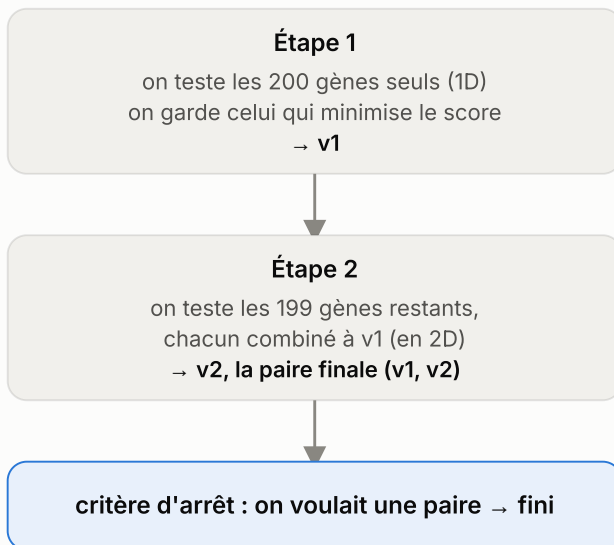
CONSTAT CENTRAL

Avec les 200 gènes aléatoires, **toutes les paires ont un Overlap > 1** (entre ~ 2.0 et ~ 2.9), pour les deux métriques — aucune n'est "bien séparée" au sens strict. C'est cohérent avec le fait que la grande majorité de 200 gènes pris au hasard ne sont probablement pas discriminants pour le type de cancer : ils ajoutent du bruit à dist_intra sans aider dist_inter . C'est exactement la motivation de la Méthode 2 : chercher activement de bonnes variables plutôt que les prendre en vrac.

Fait intéressant : les deux métriques s'accordent sur la *pire* paire (LUAD vs PRAD dans les deux cas) mais pas sur la *meilleure* (COAD-KIRC en Euclidienne, COAD-LUAD en Mahalanobis) — signe que la prise en compte des corrélations entre gènes par Mahalanobis change réellement le classement.

Sequential Forward Selection : trouver la meilleure paire de variables

Plutôt que d'utiliser les 200 gènes en vrac, on cherche activement la *paire* de gènes qui sépare le mieux les classes — pour ensuite pouvoir la dessiner en 2D. Avec 200 candidats, tester toutes les paires possibles ($200 \times 199 / 2 \approx 19\,900$) serait déjà faisable ici, mais l'algorithme utilisé — plus général — construit la paire gène par gène.



Sequential Forward Selection (SFS) — `src/variable_selection.py`. Un algorithme glouton : à chaque étape, on ajoute la variable qui améliore le plus le score, sans jamais revenir en arrière.

4.1 — Deux critères différents pour "meilleur"

L'énoncé exige deux critères distincts d'évaluation, tous deux construits sur `overlap()` :

Critère global

Moyenne d'`overlap` sur les 10 paires de classes. Cherche la paire de gènes qui distingue le mieux *les 5 classes en même temps*.

Critère local

`overlap` d'une seule paire de classes (ex. BRCA vs KIRC), calculé seulement sur les patients de ces 2 classes. Répété pour les 10 combinaisons — chacune peut avoir sa propre meilleure paire de gènes.

```
src/variable_selection.py
```

```
def global_score(X, y, cols):  
    overlap_matrix, _, _ = pairwise_overlap(X[cols], y, metric="euclidean")  
    return overlap_matrix.values[np.triu_indices_from(overlap_matrix.values, k=1)].mean()  
  
def local_score(X, y, cols, class1, class2):  
    mask = y.isin([class1, class2])  
    overlap_matrix, _, _ = pairwise_overlap(X.loc[mask, cols], y[mask], metric="euclidean")  
    return overlap_matrix.loc[class1, class2]
```

4.2 – Un bug rencontré (et corrigé)

Un `RuntimeWarning: divide by zero` est apparu : certaines paires de classes partagent la même valeur minimale sur un gène très épars, ce qui rend `dist_inter = 0` et donc l'Overlap mathématiquement infini. Corrigé en retournant explicitement `np.inf` (chevauchement total, sémantiquement correct) plutôt que de laisser Python déclencher un avertissement silencieux sur une division par zéro.

4.3 – Résultats

COMPARAISON	PAIRE DE GÈNES	OVERLAP
Toutes les classes	gene_16680, gene_11393	16.716
BRCA vs COAD	gene_4963, gene_6660	7.755
BRCA vs KIRC	gene_11393, gene_16088	1.603
BRCA vs LUAD	gene_8292, gene_11393	10.639
BRCA vs PRAD	gene_13658, gene_753	2.067
COAD vs KIRC	gene_11393, gene_9067	0.828
COAD vs LUAD	gene_12722, gene_11393	2.011
COAD vs PRAD	gene_15688, gene_18216	0.932
KIRC vs LUAD	gene_14110, gene_18216	11.195
KIRC vs PRAD	gene_18216, gene_13159	1.152
LUAD vs PRAD	gene_15688, gene_13658	1.570

DÉCOUVERTE : UN OUTLIER PEUT FAIRE "EXPLOSER" LE SCORE EN BASSE DIMENSION

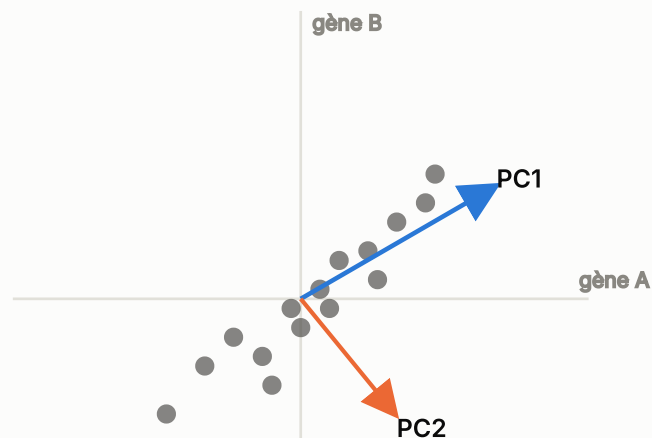
La paire globale donne Overlap = 16.7, bien pire que les ~2-3 obtenus en Méthode 1 avec 200 gènes ! Cause identifiée : gene_14110 a une valeur quasi identique chez 88.1% des patients, et quelques valeurs extrêmes chez les autres. Comme dist_intra utilise un **maximum**, un seul patient extrême suffit à gonfler la distance intra-classe quand on ne regarde que 1-2 gènes — en 200 dimensions, ce même patient est "dilué" dans la somme de 200 écarts au carré, donc moins dominant. Ce n'est *pas* la malédiction de la dimensionnalité, mais un effet distinct : la sensibilité de la formule max/min aux valeurs extrêmes.

Malgré cela, deux paires atteignent Overlap < 1 avec seulement 2 gènes bien choisis (COAD vs KIRC, COAD vs PRAD), confirmé visuellement par des nuages nettement séparés — alors que KIRC vs LUAD reste très chevauché, les deux classes s'empilant sur l'axe du gène à outliers.

5 MÉTHODE 2B – ACP

L'analyse en composantes principales : combiner plusieurs gènes en un seul axe

Au lieu de choisir 2 gènes bruts, l'ACP construit de nouvelles variables — les **composantes principales** — chacune étant une combinaison linéaire de tous les 200 gènes, choisie pour capturer le maximum de variance possible.



PC1 = direction de variance maximale
PC2 $\perp$ PC1 = variance maximale restante

L'ACP fait tourner les axes pour aligner le premier (PC1) avec la direction où le nuage de points est le plus étalé, indépendamment du fait que cette direction sépare ou non les classes.

5.1 – La formule derrière l'ACP

L'ACP diagonalise la matrice de covariance des 200 gènes standardisés : elle trouve ses vecteurs propres (les directions, appelées *loadings*) et ses valeurs propres (la quantité de variance capturée par chaque direction).

$$\Sigma = \text{Cov}(X) \quad \Sigma \cdot v_k = \lambda_k \cdot v_k$$

$$\text{variance expliquée par PC}_k = \lambda_k / \sum_i \lambda_i$$

v_k = direction de la k-ième composante principale. λ_k = variance capturée dans cette direction. On garde les 15 premières composantes ($k = 1..15$).

```
src/run_method2_pca.py
```

```
pca = PCA(n_components=15, random_state=42)
pcs = pca.fit_transform(X)                # X : 200 gènes standardisés
X_pca = pd.DataFrame(pcs, columns=[f"PC{i+1}" for i in range(15)])
# puis : exactement le même SFS que pour les gènes bruts, mais sur PC1..PC15
```

Les 15 composantes retenues expliquent **57.2%** de la variance totale des 200 gènes (12.9% pour la première composante seule). Le même algorithme SFS (section 4) est ensuite réappliqué, en remplaçant les 200 gènes candidats par les 15 composantes.

5.2 – Résultat : l'ACP n'améliore pas systématiquement

COMPARAISON	PAIRE (ACP)	OVERLAP ACP	OVERLAP BRUT
Toutes les classes	PC6, PC3	16.274	16.716
BRCA vs COAD	PC4, PC7	2.448	7.755
BRCA vs KIRC	PC2, PC12	2.808	1.603
BRCA vs LUAD	PC6, PC12	10.524	10.639
BRCA vs PRAD	PC3, PC9	2.264	2.067
COAD vs KIRC	PC2, PC3	1.510	0.828
COAD vs LUAD	PC4, PC9	3.127	2.011
COAD vs PRAD	PC3, PC7	0.801	0.932
KIRC vs LUAD	PC2, PC11	3.916	11.195
KIRC vs PRAD	PC2, PC12	1.515	1.152
LUAD vs PRAD	PC3, PC2	1.252	1.570

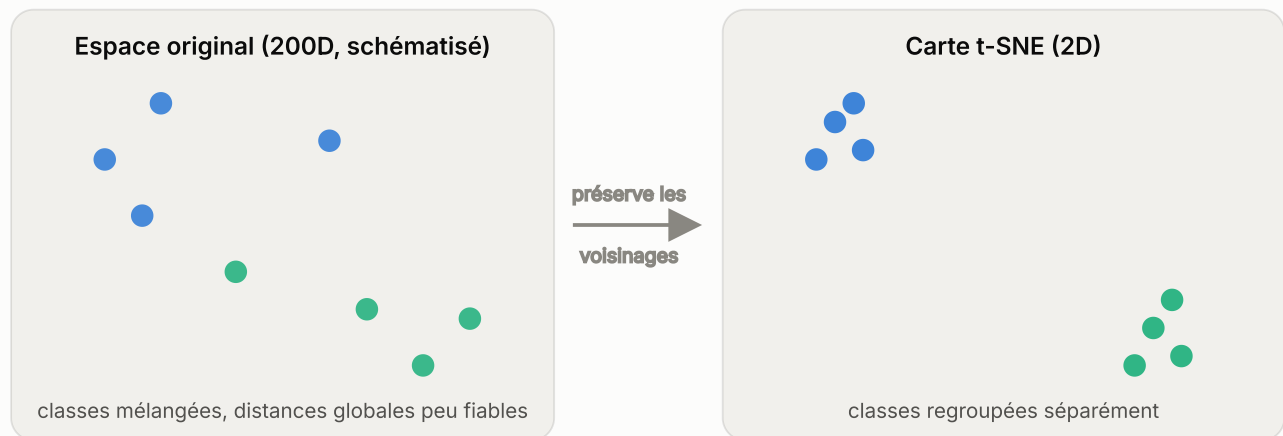
6 PAIRES S'AMÉLIORENT, 4 EMPIRENT – PAS UN PROGRÈS SYSTÉMATIQUE

L'ACP est **non supervisée** : elle maximise la variance totale sans jamais regarder les étiquettes de classe. Rien ne garantit qu'une direction de forte variance corresponde à une direction qui sépare les cancers. Hypothèse initiale testée et **infirmée** : ce n'est pas l'outlier gene_14110 qui domine une composante (son poids max est 0.088, contre 0.15–0.22 pour les gènes qui dominent vraiment PC1-PC3). L'explication retenue : l'ACP mélange des gènes très discriminants (ex. gene_11393) avec d'autres gènes à forte variance mais sans pouvoir séparateur, ce qui dilue le signal utile.

6 MÉTHODE 2C – T-SNE

Préserver les voisinages plutôt que la variance

t-SNE (t-distributed Stochastic Neighbor Embedding) est non-linéaire : au lieu de chercher des axes qui maximisent la variance (comme l'ACP), il cherche une carte en 2D où les points qui étaient *voisins* en 200 dimensions restent voisins.



t-SNE calcule, pour chaque patient, des probabilités de "voisinage" en haute dimension (affinités p) et cherche un placement en 2D dont les probabilités de voisinage (affinités q) leur ressemblent le plus.

$$p_{ij} \propto \exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)$$
$$q_{ij} \propto (1 + \|y_i - y_j\|^2)^{-1}$$

(voisinage en haute dimension, gaussienne)
(voisinage en 2D, loi de Student-t)

on ajuste $y_1 \dots y_n$ pour minimiser $KL(P \parallel Q) = \sum_{i,j} p_{ij} \cdot \log(p_{ij} / q_{ij})$

Intuition seulement : on n'a pas besoin de calculer ceci à la main, `sklearn.manifold.TSNE` s'en occupe. Le point clé à retenir : t-SNE optimise des voisinages *locaux*, pas des distances globales.

6.1 — Pourquoi il faut relancer t-SNE pour chaque paire

Comme t-SNE optimise localement (et dépend de qui est présent dans l'ensemble de points), on ne peut pas juste "extraire" une sous-partie d'un embedding global : il faut relancer l'algorithme séparément pour chaque comparaison.

```
src/run_method2_tsne.py

def run_tsne(X_subset):
    tsne = TSNE(n_components=2, random_state=42, init="pca", learning_rate="auto")
    return pd.DataFrame(tsne.fit_transform(X_subset.values), columns=["tSNE1", "tSNE2"])

# 1 fois sur les 801 patients (5 classes)      → embedding "global"
# 1 fois par paire de classes (ex. 300+146=446 patients) → 10 embeddings "locaux"
```

Chaque run utilise les 200 gènes standardisés en entrée (pas seulement 2 variables), et `Overlap()` est ensuite calculé sur l'embedding 2D résultant — uniquement pour permettre la comparaison avec les deux autres approches.

MISE EN GARDE

Les distances dans un embedding t-SNE ne sont *pas* directement comparables à des distances euclidiennes classiques — t-SNE préserve des relations de voisinage, pas des distances globales. Les valeurs d'Overlap calculées ici sont donc indicatives, moins rigoureusement interprétables qu'avec l'Euclidienne/Mahalanobis sur données brutes ou ACP.

6.2 – Résultat : nette amélioration, mais pas uniforme

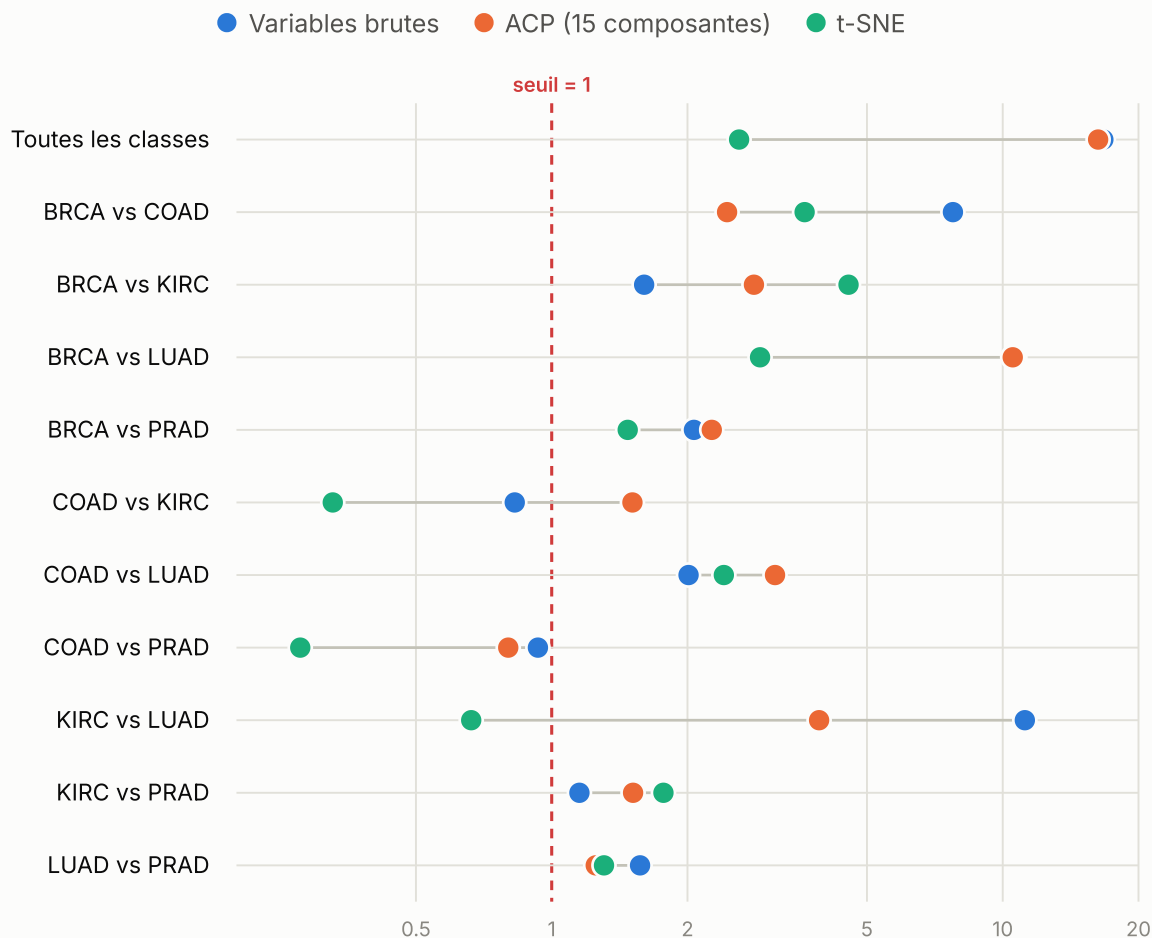
COMPARAISON	OVERLAP T - SNE	OVERLAP ACP	OVERLAP BRUT
Toutes les classes	2.603	16.274	16.716
BRCA vs COAD	3.637	2.448	7.755
BRCA vs KIRC	4.551	2.808	1.603
BRCA vs LUAD	2.897	10.524	10.639
BRCA vs PRAD	1.474	2.264	2.067
COAD vs KIRC	0.327	1.510	0.828
COAD vs LUAD	2.408	3.127	2.011
COAD vs PRAD	0.277	0.801	0.932
KIRC vs LUAD	0.663	3.916	11.195
KIRC vs PRAD	1.769	1.515	1.152
LUAD vs PRAD	1.306	1.252	1.570
Paires < 1 (bien séparées)	3/10	1/10	2/10

t-SNE l'emporte nettement sur l'embedding global (2.60 contre 16.3) et sur le cas auparavant le plus difficile (KIRC vs LUAD : 11.2 → 0.66). Mais **ce n'est pas universel** : BRCA vs KIRC est en fait *moins bien séparé* avec t-SNE (4.55) qu'avec seulement 2 gènes bruts bien choisis (1.60) — aucune des trois méthodes ne domine sur toutes les paires.

7 ANALYSE COMPARATIVE

Aucune des trois approches ne gagne partout

Le graphique ci-dessous place les 11 comparaisons sur une échelle logarithmique (les valeurs vont de 0.28 à 16.7, un facteur ~60×). La ligne verticale marque le seuil Overlap = 1 : à sa gauche, la paire est considérée comme bien séparée.



Position horizontale = valeur d'Overlap (échelle log). Ligne pointillée = seuil de séparation (Overlap = 1).
 Survolez un point pour voir sa valeur exacte.

Transformation vs non-transformation

t-SNE l'emporte nettement en moyenne et sur la séparation globale des 5 classes, mais BRCA vs KIRC est mieux séparée avec seulement 2 gènes bruts (1.603) qu'avec t-SNE (4.551). La meilleure représentation dépend donc de la paire de classes considérée, pas uniquement de la méthode choisie.

Euclidienne vs Mahalanobis (Méthode 1)

Les deux métriques mènent à la même conclusion qualitative (aucune paire bien séparée avec 200 gènes bruts) et s'accordent sur la paire la *moins* bien séparée (LUAD vs PRAD). Elles ne s'accordent *pas* sur la meilleure paire : COAD vs KIRC en tête avec Euclidienne, COAD vs LUAD en tête avec Mahalanobis (alors que cette dernière paire est presque la pire en Euclidienne) — un exemple concret de l'effet de la prise en compte des corrélations entre gènes.

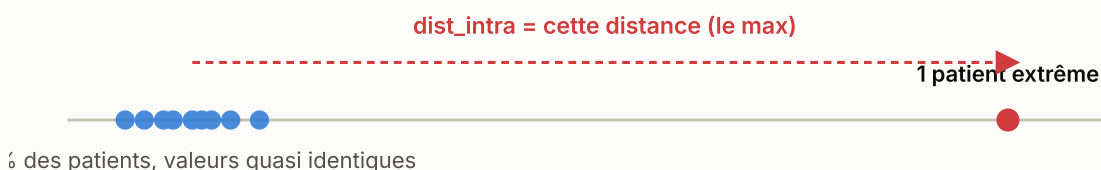
ACP vs t-SNE

t-SNE surpasse l'ACP dans la majorité des cas car il exploite les 200 gènes de façon **non linéaire** et optimise directement la préservation des voisinages (proche de la notion de séparation de classes), alors que l'ACP optimise seulement la variance totale, sans lien direct avec la séparabilité — une méthode **non supervisée** qui ignore les étiquettes de classe.

8 DISCUSSION

La limite structurelle de la mesure Overlap()

Parce que `dist_intra` utilise un **maximum** et `dist_inter` un **minimum**, la mesure reste sensible à un petit nombre de points extrêmes — quel que soit l'espace de représentation (brut, ACP, ou même t-SNE).



Cas réel observé sur `gene_14110` : un seul patient atypique suffit à multiplier `dist_intra` par un facteur énorme quand on ne travaille qu'en 1-2 dimensions, même si les 88% restants sont parfaitement compacts.

Ce même phénomène a été observé sur un embedding ACP par ailleurs visuellement bien séparé (COAD vs KIRC) : un seul patient KIRC atypique suffisait à dégrader l'Overlap alors que les deux nuages restaient visuellement distincts. Cette sensibilité est cohérente avec la mise en garde de l'énoncé : **Overlap < 1 est suffisant, mais pas nécessaire**, pour conclure à une bonne séparation — un Overlap > 1 ne prouve pas que les classes se chevauchent réellement.

9 ORGANISATION DU PROJET

Où trouver chaque morceau du code

IFT799-SD/TPs/TP1/

├─ data_cache/

| ├─ X_reduced.csv

| └─ y.csv

| └─ selected_genes.txt

└─ figures/

résultat de l'étape 0 (préparation)

801 patients × 200 gènes, standardisés

diagnostic de chaque patient

noms des 200 gènes (soumis, pas les données)

```

|   ├── method1/                tableaux intra / inter / overlap (.csv)
|   ├── method2_original_vars/  nuages de points, gènes bruts
|   ├── method2_pca/            nuages de points, ACP
|   ├── method2_tsne/          nuages de points, t-SNE
|   └── comparison_3_methods.csv tableau récapitulatif brut / ACP / t-SNE
└── src/
    ├── data_loading.py         chargement + filtrage + tirage + standardisation
    ├── build_dataset.py        enchaîne le pipeline, écrit data_cache/
    ├── distances.py            dist_intra, dist_inter, overlap, pairwise_overlap
    ├── run_method1.py          Méthode 1 : calcule et sauvegarde les tableaux
    ├── variable_selection.py   SFS + critères global / local (Méthode 2)
    ├── plotting.py             nuages de points + histogrammes en marge
    ├── run_method2_original_vars.py
    ├── run_method2_pca.py
    └── run_method2_tsne.py
└── requirements.txt

```

10 POUR LES DÉBUTANTS

Glossaire

Variance

Mesure de dispersion : à quel point les valeurs d'une variable s'écartent de leur moyenne, en moyenne au carré.

Écart-type

Racine carrée de la variance — dans la même unité que la variable d'origine, donc plus facile à interpréter.

Standardisation (score-z)

Transformer une variable pour qu'elle ait moyenne 0 et écart-type 1, afin de rendre comparables des variables mesurées sur des échelles différentes.

Covariance

Mesure de la tendance de deux variables à varier ensemble (positivement, négativement, ou pas du tout).

Matrice de covariance

Table qui donne la covariance de chaque paire de variables — la diagonale contient les variances individuelles.

Distance de Mahalanobis

Une distance qui "corrige" les corrélations entre variables, contrairement à la distance euclidienne classique qui les ignore.

Malédiction de la dimensionnalité

Quand le nombre de variables augmente, les distances entre points deviennent de moins en moins informatives si la plupart des variables sont du bruit.

ACP (analyse en composantes principales)

Technique qui construit de nouvelles variables (combinaisons linéaires des originales) ordonnées par quantité de variance capturée. Non supervisée : elle ignore les étiquettes de classe.

t-SNE

Technique de réduction de dimension non linéaire qui cherche à préserver les voisinages locaux entre points plutôt que les distances globales — souvent utilisée pour visualiser des données à haute dimension en 2D.

Sequential Forward Selection (SFS)

Algorithme glouton de sélection de variables : on ajoute, une à la fois, la variable qui améliore le plus un score choisi, sans jamais revenir en arrière.

Overlap

Indicateur numérique du TP qui compare l'étalement de deux classes (distance intra) à la distance qui les sépare (distance inter) — un score < 1 indique une séparation nette.

Sources

1. Jeu de données : UCI Machine Learning Repository, *gene expression cancer RNA-Seq Data Set (PANCAN)*, archive.ics.uci.edu/dataset/401
2. Pedregosa, F. et al., *scikit-learn: Machine Learning in Python*, JMLR 12, 2011 — `sklearn.decomposition.PCA`, `sklearn.manifold.TSNE`
3. Waskom, M., *seaborn: statistical data visualization* — inspiration pour les nuages de points avec histogrammes marginaux (`seaborn.jointplot`)